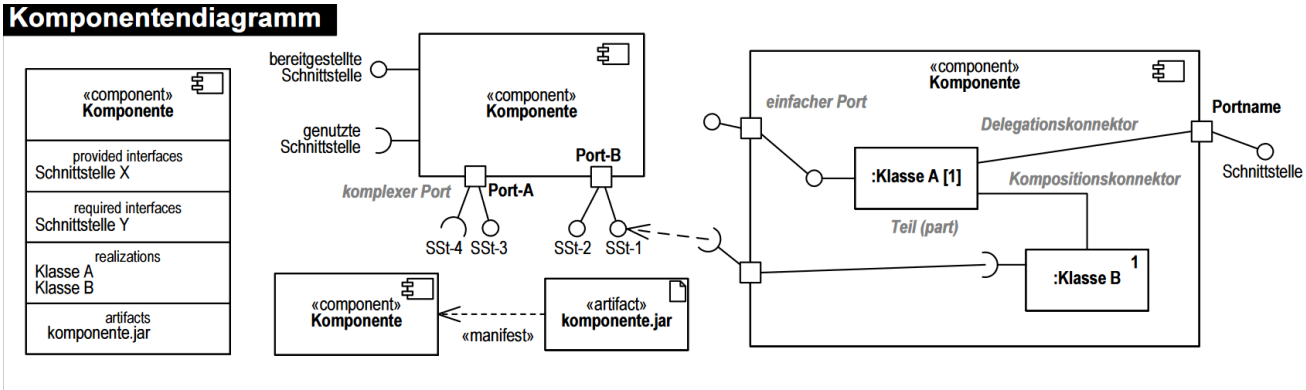
Komponentendiagramm



Zur Darstellung der verschiedenen Komponenten eines Systems und wie diese miteinander verbunden sind. Eine Komponente kann als eine abstrakter Softwarebaustein verstanden werden, welche ähnliche Funktionalität kapselt. Meistens besteht ein System aus mehreren Komponenten, welche zusammen das Gesamtverhalten umsetzten.

Wenn viele Ebenen existieren (Komponenten in Komponenten, ...) ist es übersichtlicher die verschiedenen Ebenen in separaten Diagrammen darzustellen und mit *Blackox*- und *Whitebox*-Darstellungen zu arbeiten.

> ⚠ **Wichtig!**
>
> Dies ist keine "offizielle" Beschreibung, sondern eine von mir formulierte Beschreibung anhand von Quellen und eigener Erfahrung.

Komponente

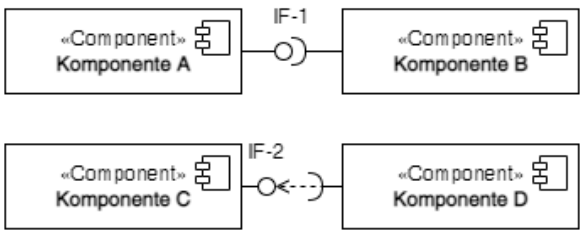
- Abstrakter Softwarebaustein.
- Kapselt möglicherweise weitere Komponenten oder Klassen.
  - *Whitebox*-Darstellung: Man sieht in die Komponente hinein
  - *Blackbox*-Darstellung: Man kann nicht sehen, was in der Komponente drin ist
- *Kompositionskonnektor*: Verwendet / Beinhaltet eine Komponente direkt
  - **Anmerkung** Ich kenne diese Notation nicht!
  - Ich denke, dass die Bedeutung ist, dass die Klasse A mit `[1]` die Klasse B mit `1` enthält.

Schnittstelle

- eine *bereitgestellte* ("Lollipop") Schnittstelle, ist eine Schnittstelle die eine Komponente anbietet, um mit ihr zu interagieren.
  - Analogie: Ein Objekt, welches verschiedene Methoden besitzt, die man aufrufen kann.

    - Hierbei wird der Name der Schnittstelle angegeben
- eine *genutzte* ("Socket") Schnittstelle gibt an, dass eine Komponente eine andere über die *bereitgestellte* Schnittstelle verwendet.

- Komponente B *benötigt* die Schnittstelle `IF-1` der Komponente A
- Komponente D ist *abhängig* von der Schnittstelle `IF-2` der Komponente C



Port

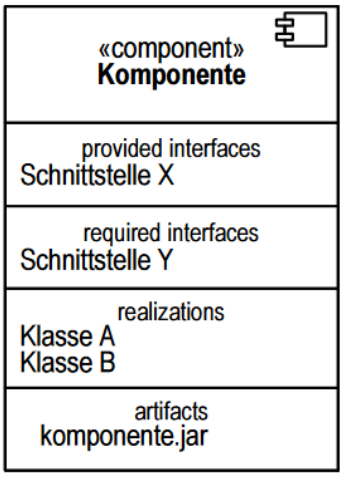
- über Ports werden interne Schnittstellen nach Außen sichtbar gemacht
    - Bei einer *bereitgestellten* Schnittstelle bekommt der Port einen "Lollipop"
    - Bei einer *genutzten* Schnittstelle bekommt der Port einen "Socket"
- *Einfacher* Port: für eine Schnittstelle
- *Komplexer* Port: für mehrere Schnittstellen
- *Delegationskonnektor*: die Verbindung einer Klasse / Komponente zu einem Port
    - Wer setzt das Verhalten um?

Artefakt

- Ein Erzeugnis einer Komponente
    - Informationen
    - Eine Datei (z.B. Log, Binary)
    - Datenbank
- `<<mainifest>>`-Beziehung: Durch wen wird das Artefakt *manifestiert* / erstellt.

Zusammenfassung als ein "Block"

- Zusammenfassung aller Informationen der Komponente
    - Welche *Schnittstellen* werde bereitgestellt?
    - Welche *Schnittstellen* werde benötigt?
    - Welche *Klassen* werde durch die Komponente umgesetzt?
    - Welche *Artefakte* werden durch die Komponente erzeugt.



Quellen

- https://developer.ibm.com/articles/the-component-diagram/
- https://www.uml-diagrams.org/component-diagrams-reference.html